CHECKING THE CHECKS: BEHAVIORAL SCREENS FOR LLM JUDGES OF FORMAL SPECIFICATIONS

Anonymous authors
Paper under double-blind review

ABSTRACT

When AI agents write their own trust artifacts, such as tests and formal specifications that establish trust in their code, those artifacts need checking at scale. In end-to-end autoformalization, for instance, agents choose their own interfaces and theorem statements, so correct answers take many forms and the reference's checks fit none. A trustworthy AI judge must then assess such trust artifacts, and every evaluation built on it inherits its errors. The standard evidence that such a judge can be trusted is agreement with human ratings, a check repeated for every new model, prompt, and task. We propose instead to collect labels once and screen every new judge with falsifiable properties that fix how scores must respond to controlled edits. Four properties target identity, bug monotonicity, missing-specification monotonicity, and stability for Lean 4 specifications scored against a gold reference. A fixed equal-weight index, $\mathrm{TI}_{\mathrm{var}}$, is the geometric mean of bug monotonicity and two variance penalties. The geometric mean acts as a smooth conjunction, so no strong component can offset a weak one. On a 72-pair validation set, the index tracks judge-level human agreement across three model families with eight configurations each. For the Codex panel, the Spearman correlation is 0.833 on averaged labels and 0.905 and 0.738 on the two rating passes. Bug-ladder checks expose a deliberately lexical judge that matches the semantic prompts' human agreement yet reverses on 20 of 24 ladders. Judge validation thus moves from a per-judge annotation cost toward a one-time human panel plus automated, falsifiable checks that any new judge must pass.

1 INTRODUCTION

AI agents now write the trust artifacts for their own code: the tests (Haque et al., 2026) and formal specifications (Yang et al., 2024) on which trust in that code rests. A formal specification states the guarantees a program should satisfy, such as returning a sorted permutation of its input. To trust an AI-written trust artifact, one must check the artifact and check whatever judge performs that check. We start with formal specifications because the proof assistant fixes what each theorem statement means, leaving one question: whether the statements capture the intent.

In end-to-end autoformalization, an agent receives source code and writes the Lean 4 implementation, tests, theorem statements, and proofs without a scaffold (Miranda et al., 2026). Correct solutions take many forms, because none need share the reference's interfaces, data representations, or decomposition into theorem statements. Two questions then decide whether a solution is right: do its proofs establish its own theorem statements, and do those statements express the reference's guarantees? Lean settles the first. It cannot settle the second, because a proof establishes the theorem that was stated, not that the stated theorem covers the intent (Lahiri, 2024). Rerunning the reference's tests and theorems cannot settle it either: they fail against a different representation whether or not the solution is correct (Miranda et al., 2026). Translating code into another language raises the same mismatch whenever the translation chooses its own interfaces. What remains is a comparison of meaning between the candidate's and the reference's specifications. Large language models (LLMs) already make that comparison at scale (Miranda et al., 2026; Lin et al., 2025). Every evaluation built on such a judge inherits its errors, so the judge's reliability decides what the benchmark measures.

How do we know that such a judge can be trusted? The standard evidence is agreement with human ratings: a judge counts as reliable once its scores correlate with expert annotations (Chen et al., 2026)

or rubric-based ratings (Liu et al., 2023; Zheng et al., 2023). On objective tasks, consistency across repeated calls serves as further evidence (Schroeder & Wood-Doughty, 2024). Agreement varies across models, tasks, and rated properties, so evidence for one judge configuration does not carry over to the next (Bavaresco et al., 2025). The check is therefore repeated for every new model, prompt, and task, and each repetition costs expert annotation. Reusing one labeled set is cheaper, but selecting many configurations against the same labels risks adaptive overfitting (Dwork et al., 2015). A new artifact distribution also raises the separate question of whether old labels remain representative.

We propose instead to collect labels once and to screen every new judge with falsifiable properties. A property fixes, by construction, how a judge's score must respond when a controlled edit preserves or damages a reference artifact. A copy of the reference must score the maximum, accumulating bugs must not raise the score, and renaming a bound variable must not change it. Checking these responses costs judge calls, not human labels. A failed property names a concrete behavior that human agreement can hide; a pass is evidence about the tested behaviors only. The one-time panel links properties to the human criterion: we check whether judges with better property behavior agree more with humans, then freeze the screen.

We instantiate this screen for Lean 4 formal specifications judged against a gold reference (Figure 1). Four properties target identity, bug monotonicity, missing-specification monotonicity, and stability across repeated calls. A fixed equal-weight index, $\mathrm{TI}_{\mathrm{var}}$, summarizes bug monotonicity and two variance penalties as a geometric mean. The geometric mean is a smooth conjunction, so no strong component can offset a weak one. The per-property report stays alongside the scalar.

On a 72-pair validation set, the index tracks judge-level human agreement across three model families with eight configurations each. For the GPT-5.3-Codex family, the Spearman correlation is 0.833 on averaged labels, with a 95% bootstrap interval of $[0.24, 1.00]$. These estimates are optimistic, because the same labels informed the index form, the prompts, the repeat aggregation, and the threshold. The screen catches what human agreement misses: a deliberately lexical judge matches the semantic prompts' human agreement yet reverses on 20 of 24 bug ladders.

Judge validation thus moves from a per-judge annotation cost toward one human panel plus automated checks. Transfer to new judges and domains remains to be tested (Section 6). Our contributions are:

- A screening protocol: one human validation set plus four falsifiable properties that screen later judge configurations without new labels (Section 4).
- An instantiation for Lean 4 formal specifications: scripted probes, a fixed equal-weight index, and a per-property failure report (Section 4).
- Evidence on three judge families: the index tracks validation-set human agreement, a lexical judge passes human agreement but fails the screen, and the scalar alone is not a rejection rule (Section 5.3).

2 Related Work

Validating LLM judges through human agreement. G-Eval (Liu et al., 2023) and MT-Bench (Zheng et al., 2023) establish a judge's reliability by its agreement with human ratings, so each new judge or task calls for the same comparison. Bavaresco et al. (2025) measure that agreement across 20 tasks and find it task dependent, which is why the comparison cannot be made once for all judges. Schroeder & Wood-Doughty (2024) measure consistency across repeated judgments, which detects instability but not a consistently wrong judge. We keep human agreement as the criterion, collect it once, and add checks whose required direction is known without labels.

AI judges for AI-written trust artifacts. ICE-Score (Zhuo, 2024) and CodeJudge (Tong & Zhang, 2024) score generated code with a language model where execution gives incomplete evidence, and validate the judge by its agreement with human or execution-based labels on their own tasks. SWT-Bench validates agent-written tests by execution against the developers' fix (Mündler et al., 2024), which requires the test and the reference to share one interface, the condition that end-to-end formalization breaks. For formal statements, FormalAlign learns an alignment score between informal and formal statements (Lu et al., 2025), whereas our target is the behavior of a judge comparing candidate and gold specifications. Prover pipelines filter autoformalized statements with

an LLM judge (Lin et al., 2025; Wang et al., 2025). REFORM validates its statement judge against expert annotations (Chen et al., 2026), evidence that does not transfer to the next judge configuration. VERIBENCH scores agent-written Lean specifications with an LLM judge (Miranda et al., 2026); we study when such judges can be trusted. Each of these judges answers a question that execution or proof checking cannot settle: whether an artifact means the right thing.

Behavioral tests with a known direction. Metamorphic testing checks a program without an oracle by fixing how its output must change under a transformation of its input (Chen et al., 1998). The test-oracle problem it addresses (Barr et al., 2015) is the one an AI-written trust artifact raises: running the artifact cannot tell whether the artifact is right. CHECKLIST applies the same idea to NLP models with templated invariance and directional tests, and reveals failures in models with high held-out accuracy (Ribeiro et al., 2020). Our properties are metamorphic relations on the judge: identity, ladder monotonicity, and rewrite invariance fix the required response to a scripted edit of a gold artifact. The one-time human panel adds what metamorphic testing alone lacks, a measurement of whether the relations track the human criterion across judges.

3 PROBLEM SETUP

Trust artifacts, intent, and formal specification. An item consists of an informal problem statement, a gold Lean 4 artifact g, and a candidate artifact x written by an agent. We call the informal requirement the *intent*. An artifact is a Lean source file with an implementation and, potentially, theorem statements, predicates, unit tests, and proofs; its theorem statements and tests are its trust artifacts. Its *formal specification* is the set of theorem statements about the implementation together with the predicates they use. A sorting specification, for example, can require both ordered output and preservation of the input elements. A proof establishes a theorem under its stated assumptions; it does not determine whether that theorem captures the intent. A unit test checks one concrete case and is not part of the specification in our terminology (Appendix F). We use "specification" in this sense whether an agent, a person, or a script wrote it.

What the judge must decide. The target is *reference-based specification quality*: how completely and correctly the candidate specification captures the guarantees of the gold reference. A candidate can type-check yet omit a requirement, impose an unintended assumption, or assert a vacuous property. The judge prompts ask which reference guarantees the candidate captures and assign zero to trivial statements such as `True` (Appendix A). This instruction does not establish that judges detect every such failure. When a candidate shares the reference's interface, automated proof attempts in both implication directions can sometimes establish equivalence with the reference specification (Lahiri, 2024; Poiroux et al., 2025). Such attempts do not decide equivalence in general, and a failed attempt does not show inequivalence. Across differing representations they need an explicit correspondence, which end-to-end generation does not supply. A gold reference gives the judge a concrete comparison target, but it does not make the judge immune to strategic manipulation (Robertson & Koyejo, 2025).

Judge configuration. A judge configuration is $\theta = (\text{model}, \text{prompt}, n_{\text{repeats}}, \text{aggregation})$ with score $J_\theta(x, g) \in [0, 1]$. The headline panels take the median over repeated calls. Despite the prompt label "REFERENCE IMPLEMENTATION," the reference field holds the gold file's final theorem block, and the candidate field holds the full candidate file. The measured task therefore depends on this extraction rule as well as on the nominal rubric, and a changed rule requires a new assessment.

Two levels of agreement. *Artifact-level agreement* is the correlation between one judge's scores and the human ratings over candidate–gold pairs. *Judge-level association* is the correlation, across judge configurations, between their artifact-level agreement and a property index. The first says how well one judge tracks humans; the second says whether the screen orders judges the way humans would.

4 SCREENING JUDGES WITH FALSIFIABLE PROPERTIES

Labels once, checks for every judge. Figure 1 and Table 1 lay out the protocol. A human panel rates candidate–gold pairs once; this validation set links judge behavior to the intended scoring

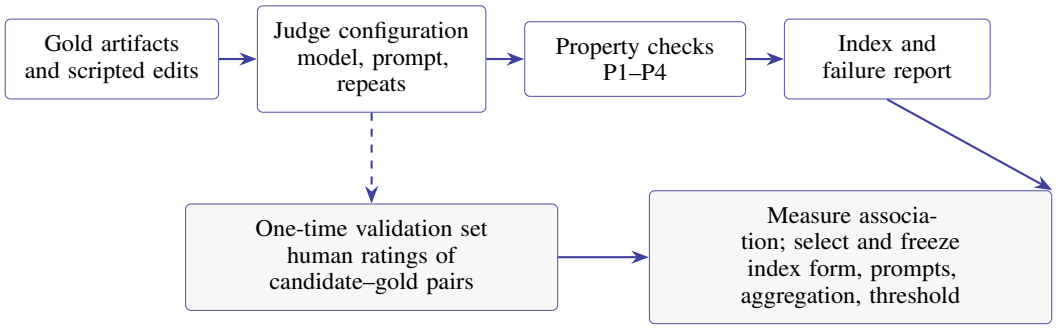


Figure 1: Human labels enter once (bottom); every later judge faces only the automated checks (top). The bottom path measures validation-set association and selects protocol choices, so its results are optimistic rather than a held-out test.

Stage	Human labels	Judge computation
Establish the screen	Rate candidate–gold pairs once	Measure human agreement and property responses
Select and freeze	Use the validation ratings	Choose index form, prompts, aggregation, threshold
Screen a new judge	Do not consult the labels	Run the frozen probes; inspect index and failure report
Test transfer	Collect independent ratings under the frozen rubric	Evaluate the frozen choices on untouched items

Table 1: Screening a new judge costs inference, not labels; establishing transfer needs a separate, locked test.

task. For each judge configuration, we measure agreement with those ratings and, separately, its responses to scripted edits of gold artifacts. The edits require judge calls but no new labels. We use the validation set to choose the index form, the prompts, the repeat aggregation, and the decision threshold, then freeze them. A later configuration faces the same frozen checks without access to the labels. The present evidence supports this as a screen; it does not establish transfer to an unseen configuration or artifact distribution (Section 6).

Four falsifiable properties. Each property fixes, by construction, how J_θ must respond to a scripted edit of a gold artifact g. **P1 (identity)** requires the maximum score when an artifact is compared with itself or with a meaning-preserving rewrite of itself, $J_\theta(g, g) = 1$. The rewrites include alpha-renaming, which consistently changes bound-variable names without changing their meaning. In the recorded panels, P1 is assessed through four rewrite probes per artifact (renaming, reordering, style, and alpha-renaming); no literal self-comparison was scored. **P2 (bug monotonicity)** requires non-increasing scores as bugs accumulate in the candidate. The historical bug ladder used here edits the candidate text, including theorem statements; it is not restricted to implementation bugs. **P3 (missing-specification monotonicity)** requires non-increasing scores as theorem statements and unit tests are removed. We keep this historical name although unit tests are not specifications under Section 3. Each ladder level extends the previous one, so the required ordering concerns cumulative damage, not equal severity of individual edits. Ties are allowed; any local score increase is a reversal. **P4 (stability)** requires stable scores across repeated calls on the same input. The properties are targets for the probes, not theorems about every scripted transformation; Appendix E records where the historical generators fall short.

A fixed equal-weight summary. Different failures need different checks, so the protocol keeps the per-property report. For ranking configurations, we also use one scalar, $\mathrm{TI}_{\mathrm{var}}$, computed from bug-ladder scores only:

$$\mathrm{TI}_{\mathrm{var}} = \exp\left[\tfrac{1}{3}\log C_f + \tfrac{1}{3}\log(1-\widetilde{V}_c) + \tfrac{1}{3}\log(1-\widetilde{V}_s)\right]. \tag{1}$$

Here C_f is the mean transformed per-file Spearman correlation between bug level and score, V_c is the variance of these per-file values, and V_s is the mean within-level variance across repeated calls. The tildes denote min–max normalization across the configurations of the same family panel, and the implementation floors each logarithm's argument at 10^{-6} (Appendix B). The geometric mean acts as a smooth conjunction: a strong component cannot fully offset a weak one, and equal weights avoid fitting component weights to the human ratings. The scalar omits P1, P3, and a direct penalty for local reversals, so it summarizes selected behavior rather than replacing the report. Its panel-dependent normalization also means that its absolute value can change when the comparison panel changes. Prospective use therefore needs a frozen normalization reference, whose transfer we have not tested.

5 Experiments

5.1 Validation set

We evaluate candidate–gold pairs from VeriBench (Miranda et al., 2026). One annotator supplied holistic ratings on a 0–5 scale in two passes, pass1 under a stricter rubric and pass2 under a more lenient one. The collection covered 75 pairs from 25 problems with three agent generations per problem. One gold filename could not be resolved, leaving 72 analyzed pairs from 24 problems. We report each pass and the archived averaged-label view; they are two judgments by one person, not independent annotators. Inter-pass agreement is Spearman 0.249, quadratic-weighted $\kappa = 0.372$, and mean absolute error 0.771 on the shared pairs (Appendix D). These statistics describe label disagreement; they do not establish an upper bound on judge agreement. The original written rubric was not recovered, so we cannot reconstruct every annotation instruction.

This panel is the *validation set*: it informed index-form selection, repeat aggregation, the threshold, and the prompt panel, so correlations on it are optimistic. The fixed equal-weight $\mathrm{TI}_{\mathrm{var}}$ has no fitted weights, so there is no training set. A separate weight-tuned comparison uses these same pairs, and no analysis in this paper is a test-set result. For each judge, artifact-level agreement is the Spearman correlation between its scores and the human ratings on the analyzed pairs. We then ask whether the index orders configurations by that agreement.

5.2 Property checks on GPT-5.3-Codex

Figure 2 summarizes the 24-problem panel for the three semantic Codex prompts. Each bug ladder has five levels, the original artifact plus four cumulative edits, and each artifact has four rewrite probes. Mean scores fall from 1.0 at level zero to 0.24–0.27 after the first cumulative edit and to about zero thereafter. Averages conceal local reversals: the strict prompt reverses on four of 24 ladders, whereas the baseline and checklist prompts have none (Table 3). Renaming, reordering, and style rewrites leave mean scores at or near 1.0. The archived alpha probes are the exception, but their generator can produce malformed Lean text. The baseline and checklist prompts assign them a mean score of zero; the strict prompt assigns 0.179. Appendix E keeps the per-configuration diagnostics and the corresponding Claude Opus curves.

Alpha-renaming and a canonicalization proxy. The historical alpha routine can duplicate the assignment marker and rename signature variables without updating the proof body. Its score drops therefore cannot establish a failure of invariance under valid alpha-renaming. A post-hoc proxy replaces each judge's alpha scores with its median score under style edits. Across the eight Codex-panel configurations, this lowers the mean per-judge standard deviation across identity-probe scores from 0.250 to 0.070. This manipulation changes recorded scores rather than the judge's inputs. A real fix must rescore canonicalized text and verify that the transformation preserves meaning.

Coverage of P3. The eight-configuration panels contain rewrite probes and bug ladders but no missing-specification ladders. They therefore cannot say which association P3 has with human agreement. P3 remains a proposed part of the screen; none of the bug-ladder statistics below belongs to it.

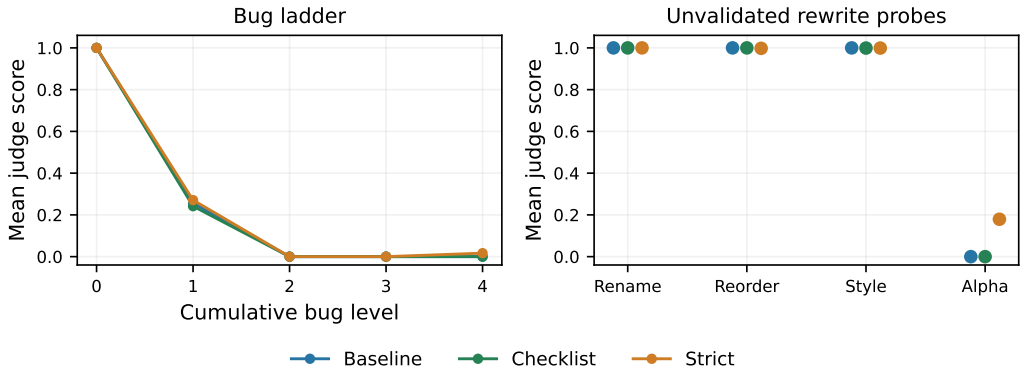


Figure 2: Early bug edits and malformed alpha probes lower the Codex judges' mean scores. Each point averages median scores over 24 problems, with three calls per problem and prompt; these are descriptive means, not uncertainty intervals. Local reversals remain visible in Table 3.

	Averaged labels				$\mathrm{TI}_{\mathrm{var}}$ by pass	
Family	$v1$	$v2$	$\mathrm{GM}_{.4,.6}$	$\mathrm{TI}_{\mathrm{var}}$	pass1	pass2
Codex	0.762	0.476	0.857	0.833	0.905	0.738
Claude Opus	0.429	0.143	0.714	0.643	0.810	0.595
Qwen3-Coder	0.524	0.500	0.786	0.905	0.476	0.857
Codex $\mathrm{TI}_{\mathrm{var}}$: 95% interval				$[0.24, 1.00]$	$[0.41, 1.00]$	$[0.11, 1.00]$
Codex $\mathrm{TI}_{\mathrm{var}}$: permutation p				0.017	0.005	0.045

Table 2: The fixed index is competitive with a two-component geometric comparison whose weights were selected on pass1. Entries are Spearman correlations between each index and judge-level human agreement on the validation set, with eight configurations per family. Codex intervals and permutation probabilities use 10,000 resamples and permutations, seed 42; exact permutation probabilities for every cell are in Appendix C.4. Appendix B defines all indices.

5.3 The index tracks judge-level human agreement on the validation set

The three primary families are GPT-5.3-Codex, Claude Opus 4.7, and Qwen3-Coder. Each eight-configuration panel contains three semantic prompts, a deliberately lexical prompt on the same model, two GPT-4o-mini prompts, edit distance, and a random control. These are family panels with shared controls, not eight independent models per family. The frozen model identifiers and complete main-panel prompts appear in Appendix A. Scores are medians over three repeated calls for language-model judges; the Codex-panel controls use nine repeats, and the other panels use three.

Figure 3 uses the fixed equal-weight index of Equation 1. Its judge-level Spearman association is positive for each primary family and label view (Table 2). For Codex, the averaged-label Pearson and Kendall associations are 0.868 and 0.643. The Codex bootstrap intervals are wide, and the resampling does not correct for the earlier protocol selection. Exact permutation probabilities over all 8! orderings are 0.096, 0.022, and 0.132 for Claude Opus and 0.005, 0.243, and 0.011 for Qwen3-Coder on averaged, pass1, and pass2 labels (Appendix C.4). Only some cells are individually distinguishable from a random ordering at the 0.05 level, and the nine tests are unadjusted. Pass-to-pass differences also matter: Qwen's association changes from 0.476 on pass1 to 0.857 on pass2. Removing the two non-LLM controls leaves six configurations per family, with averaged-label associations of 0.771 for Codex, 0.486 for Claude Opus, and 0.943 for Qwen3-Coder. Among the four same-model prompts alone, the averaged-label associations are 0.400, 0.000, and 0.800, so the index orders semantic prompts coarsely at best. With eight configurations and disagreeing label passes, these estimates support a descriptive association rather than a precise or portable effect size.

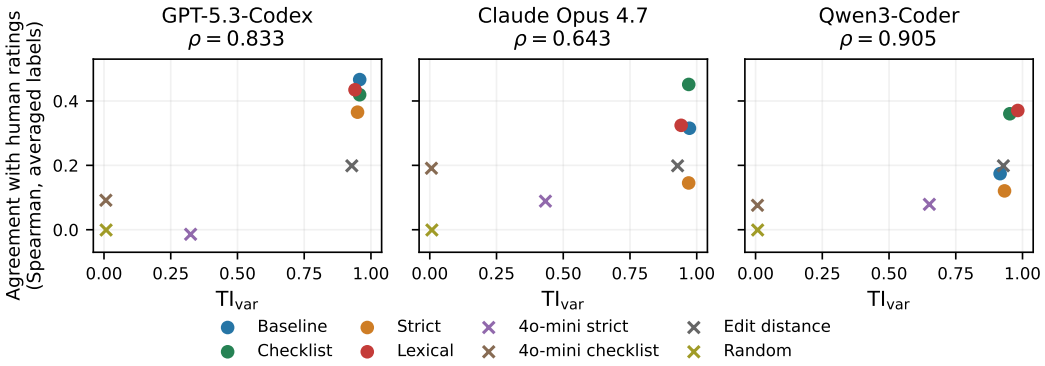


Figure 3: The fixed equal-weight index has a positive association with human agreement in each primary family, but high index values do not rule out lexical judges. Each point is one configuration; the vertical axis is its Spearman agreement with averaged human labels on 72 pairs. Panel titles report the correlation across eight configurations; Table 2 gives Codex uncertainty.

Codex-panel configuration	Human ρ (avg)	TI_{var}	Ladders with reversals
Baseline prompt	0.466	0.958	0/24
Checklist prompt	0.419	0.958	0/24
Strict prompt	0.365	0.950	4/24
Lexical prompt	0.435	0.940	20/24
GPT-4o-mini, strict	−0.014	0.324	18/24
GPT-4o-mini, checklist	0.092	0.007	16/24
Edit distance	0.199	0.928	24/24
Random	−0.001	0.008	24/24

Table 3: The lexical judge agrees with humans despite frequent bug-ladder reversals, and the scalar index penalizes it only slightly. A reversal is any local score increase as cumulative edits are added; the diagnostic uses the recorded repeat-aggregated scores.

5.4 Human agreement alone can miss a bad judge

The lexical prompt, `BAD_STRINGY`, instructs the model to ignore theorem meaning and grade textual overlap. On the Codex panel, its human agreement of 0.435 lies among the semantic prompts' values, 0.365–0.466. A validation protocol that stops at human agreement would accept it. Yet it reverses on 20 of 24 bug ladders (Table 3). The scalar does not expose it either: its index value is 0.940, below the three semantic prompts but far above the failing GPT-4o-mini configurations, and edit distance scores 0.928. In the Qwen panel, the lexical configuration even has the highest index value, 0.982. The failure report therefore carries information that both human agreement and the scalar can miss. No validated cutoff on TI_{var} establishes that a judge is acceptable; the screen's verdict is the per-property report.

Choice of index. The two-component geometric comparison is numerically higher than TI_{var} for Codex and Opus on averaged labels (Table 2). It uses bug monotonicity and decision stability with weights 0.4 and 0.6, selected on pass1, rather than fixed equal weights. We retain TI_{var} for its explicit variance penalties and positive associations across the primary families and passes, not because these data establish its superiority. The bug-monotonicity component alone has averaged-label associations of 0.643, 0.467, and 0.690, so the two variance penalties raise the association in eight of the nine family–pass cells (Appendix C.4). The exception is Qwen pass2, where the component alone reaches 0.905 and the full index 0.857. A separate search over six component weights gives cross-pass associations of 0.810 and 0.786 on the same pairs (Appendix C). The pass1-selected geometric form reaches 0.769 on an expanded 13-configuration panel, with a 95% interval of $[0.34, 0.95]$ (Appendix D.2). Neither comparison provides a held-out estimate. A separate Leanstral panel has

Measured diagnostic	pass1	pass2	avg
Mean identity-rewrite score	−0.21 (0.62)	0.02 (0.98)	−0.02 (0.98)
Bug-rank monotonicity	0.60 (0.13)	0.67 (0.08)	0.71 (0.06)
Bug weak monotonicity	0.63 (0.11)	0.78 (0.03)	0.75 (0.04)
Repeat decision stability	0.47 (0.24)	−0.23 (0.57)	0.14 (0.76)

Table 4: Bug weak monotonicity has the largest observed association among the measured diagnostics; this table contains no P3 missing-specification result. Entries are Spearman correlations with human agreement across eight Codex-panel configurations; unadjusted permutation probabilities appear in parentheses.

averaged-label association -0.095 for the fixed index (Appendix D.3), so the positive result does not hold for every recorded family.

Weight and normalization sensitivity. Recomputing the fixed index on a local grid of weight triples gives averaged-label Spearman ranges of 0.738–0.857 for Codex, 0.643–0.714 for Opus, and 0.881–0.976 for Qwen. This analysis uses only committed component vectors and keeps their normalization fixed (Appendix C.3). It does not assess sensitivity to changing the normalization panel. We do not report leave-one-family-out transfer: the panels share controls and normalize separately, so pooling their scores would not constitute an independent family test.

Which measured statistics matter. Among the recorded diagnostics, weak monotonicity on bug ladders has the largest association with human agreement on each label view (Table 4). It measures the absence of local increases; bug-rank correlation measures the overall direction. The archived field name `P3_weak_mono` refers to this bug statistic, not to the P3 missing-specification property. Identity-rewrite scores and repeat decision stability have weaker or inconsistent associations. Only the weak-monotonicity statistic on pass2 and averaged labels has an unadjusted permutation $p < 0.05$; these component comparisons are exploratory.

Artifact-score threshold. A historical sweep of the artifact-score threshold over six values from 0.60 to 0.85 lowers the observed decision-flip rate from 0.058 to 0.046 and the pass rate from 0.583 to 0.536 (Appendix Table 6). The selected threshold is 0.85, which has the lowest observed flip rate in that sweep, although its interval overlaps those of every other threshold. This thresholds $J_\theta(x, g)$, not the judge-level index, and measures repeatability rather than correctness of the accepted artifacts.

6 Limitations and a Locked Test

Validation evidence, not a test. The same labels informed the index form, repeat aggregation, threshold, and prompt panel, so the reported correlations are optimistic validation-set estimates. They rest on one annotator and eight configurations per family, with shared controls rather than independent samples of eight models. No test-set result establishes reliability for a new judge or artifact distribution. We also collected 343 labels from five raters over 78 candidate–gold pairs. That pool uses statement coverage with a theorem-spam deduction, a different rubric from the panel's holistic ratings. Its Spearman agreement with the panel labels is approximately 0.13 on 63 shared pairs. Two pre-registered screens asked whether property indices tracked agreement with that pool. The screen of 14 configurations was underpowered; the expanded screen of 32 found no index meeting its registered criterion. Its largest absolute Spearman association was -0.39 with Holm-adjusted permutation $p = 0.25$; leave-one-out selection transfer yielded $\rho = 0.24$, $p = 0.49$. These negative results do not demonstrate transfer under the original criterion, nor can the pool serve as a test set for that criterion. Appendix D.3 records the corresponding negative-control evidence.

What the screen covers. The probes do not establish sensitivity to vacuity, subtle weakening, over-constraint, or formatting-only edits beyond the four rewrites studied here. Constructing probes with known directions for these failures is the next methodological step; we claim no result for them. The historical alpha generator can produce malformed text, so its low scores do not establish sensitivity to valid alpha-renaming. Other scripted rewrites and the intended ordering of bug edits also lack semantic

validation. All configurations are single-call rubric prompts, so structured multi-stage judges (Tong & Zhang, 2024) are untested. No automated equivalence baseline on interface-matched pairs was run (Section 3). We cannot separate rubric ambiguity from misunderstanding Lean semantics with the current annotations. Presentation, including redundant theorems, implementation bloat, and unnecessary abstraction, is unassessed; holistic human ratings may reflect it implicitly. We did not optimize candidates against the judge, so resistance to gaming is untested.

Scope. All candidates and probes come from VERIBENCH in Lean 4. The protocol could be instantiated for another proof assistant or for AI-written tests, but neither cross-assistant nor cross-benchmark transfer has been tested. A score is relative to the supplied gold reference and extraction rule, not a guarantee that all intended requirements are present. Reference-free screening and the number of annotators needed for a stable criterion remain open. The index also depends on its comparison panel and omits constant ladders with undefined correlations; these implementation choices limit portable screening.

A locked test. A decisive test would first freeze the rubric, property generators, score extraction, aggregation, index, normalization reference, and any rejection rule. Independent raters would then label new candidate–gold pairs under that rubric, with new judge configurations fixed before their labels are examined. The analysis would report both artifact-level agreement and judge-level association, including uncertainty and property-specific failures. A separate distribution-shift test would be needed for new domains. Because the original rubric text is unavailable, recovering it or validating an explicitly reconstructed criterion is a prerequisite to claiming a matched-rubric test.

7 CONCLUSION

Agents now write the trust artifacts for their own code, and a language-model judge decides whether those artifacts say the right thing. We replaced the per-judge human validation of that judge with one human panel and four falsifiable properties that fix how scores must respond to controlled edits. On the validation set, the fixed equal-weight index tracks judge-level human agreement across three families, and the property report exposes a lexical judge that human agreement rates as acceptable. The report, not the scalar, is the screen's verdict, and a locked test must still establish transfer beyond the labels used to select the protocol. Human judgment then becomes a reusable foundation: collected once, it anchors automated, falsifiable checks that every new judge must pass.

Reproducibility Statement

The anonymous supplement contains frozen score-level inputs and a standalone analysis script. It reconstructs the fixed-index associations, local weight sensitivity, and recorded property diagnostics without new model inference. The judge-level diagnostics of Appendix C.4 are exact recomputations from the same frozen vectors. Appendix A gives the prompt templates, exact model identifiers, input extraction, and repeat aggregation used for the main panels. Appendix B specifies the indices, including normalization and numerical edge cases. The supplement's manifest records input hashes, and its README identifies the reproduced outputs and exclusions. It does not regenerate the original model calls, bootstrap intervals, human annotations, or every historical appendix analysis. The original human-panel rubric text was not recovered, which limits exact replication of the annotation process.

Ethics Statement

An author supplied the human ratings used for the headline validation analysis. The study evaluates generated Lean code and formal statements, without personal data in the judged artifacts. It makes no deployment claim and does not establish that screened judges can replace expert review in consequential settings.

Artificial Intelligence Use Statement

LLMs generated candidate Lean artifacts and served as the judges under study. Coding assistants helped implement experiments and analysis scripts, check and interpret results, and prepare figures. They also assisted with methodological feedback, literature retrieval, manuscript drafting, and editing under author supervision. The human ratings used for validation were supplied by a human annotator, not generated by these tools. The authors are responsible for the manuscript's claims, citations, and interpretation of the recorded results.

References

Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015. doi: 10.1109/TSE.2014.2372785.

Anna Bavaresco, Raffaella Bernardi, Leonardo Bertolazzi, Desmond Elliott, Raquel Fernández, Albert Gatt, Esam Ghaleb, Mario Giulianelli, Michael Hanna, Alexander Koller, André F. T. Martins, Philipp Mondorf, Vera Neplenbroek, Sandro Pezzelle, Barbara Plank, David Schlangen, Alessandro Suglia, Aditya K. Surikuchi, Ece Takmaz, and Alberto Testoni. Llms instead of human judges? a large scale empirical study across 20 nlp evaluation tasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 238–255. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-short.20. URL https://aclanthology.org/2025.acl-short.20/.

Guoxin Chen, Jing Wu, Xinjie Chen, Wayne Xin Zhao, Ruihua Song, Chengxi Li, Kai Fan, Dayiheng Liu, and Minpeng Liao. ReForm: Reflective autoformalization with prospective bounded sequence optimization. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/419bd6a89ab56e9c09ededb560b06192-Abstract-Conference.html.

T. Y. Chen, S. C. Cheung, and S. M. Yiu. Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, Department of Computer Science, The Hong Kong University of Science and Technology, 1998. URL https://arxiv.org/abs/2002.12543. arXiv:2002.12543.

Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638, 2015. doi: 10.1126/science.aaa9375. URL https://doi.org/10.1126/science.aaa9375.

Sabrina Haque, Sarvesh Ingale, and Christoph Csallner. Do autonomous agents contribute test code? a study of tests in agentic pull requests, 2026. URL https://arxiv.org/abs/2601.03556.

Shuvendu K. Lahiri. Evaluating LLM-driven user-intent formalization for verification-aware languages. In *Proceedings of the 24th Conference on Formal Methods in Computer-Aided Design (FMCAD)*, pp. 142–147. TU Wien Academic Press, 2024. doi: 10.34727/2024/isbn.978-3-85448-065-5_19. URL https://arxiv.org/abs/2406.09757.

Yong Lin, Shange Tang, Bohan Lyu, Jiayun Wu, Hongzhou Lin, Kaiyu Yang, Jia Li, Mengzhou Xia, Danqi Chen, Sanjeev Arora, and Chi Jin. Goedel-prover: A frontier model for open-source automated theorem proving, 2025. URL https://arxiv.org/abs/2502.07640.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: Nlg evaluation using gpt-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 2511–2522. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.153. URL https://arxiv.org/abs/2303.16634.

Jianqiao Lu, Yingjia Wan, Yinya Huang, Jing Xiong, Zhengying Liu, and Zhijiang Guo. FormalAlign: Automated alignment evaluation for autoformalization. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/fceedf8c9c0ff51f41b9fe0294ef0070-Abstract-Conference.html.

Brando Miranda, Srivatsava Daruru, Ethan S Hersch, Zhanke Zhou, Allen Nie, Daneshvar Amrollahi, Leni Aniva, Iddah Mlauzi, Kirill Acharya, Elyas Obbad, Dilara Soylu, Weston Kirk, Zixiao Jolene Wang, Kai Fronsdal, Ying Li, Donald Poindexter Jr, Rakshit Kaushik, Shurui Liu, Yegor Denisov-Blanch, Steven Dillmann, Simon Obstbaum, Santiago Cuellar, John Sarracino, Rylan Schaeffer, Mo Tiwari, Donghyun Lee, Bo Han, and Sanmi Koyejo. Veribench: An end-to-end formal verification benchmark for AI coding agents in lean 4. In *3rd AI for Math Workshop: Toward Self-Evolving Scientific Agents*, 2026. URL https://openreview.net/forum?id=lkL0qnUv3p.

Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. SWT-bench: Testing and validating real-world bug-fixes with code agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2406.12952.

Auguste Poiroux, Gail Weiss, Viktor Kunčak, and Antoine Bosselut. Reliable evaluation and benchmarks for statement autoformalization. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 17947–17969. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.907. URL https://aclanthology.org/2025.emnlp-main.907/.

Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 4902–4912. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.442. URL https://aclanthology.org/2020.acl-main.442/.

Zachary Robertson and Sanmi Koyejo. Let's measure information step-by-step: AI-based evaluation beyond vibes, 2025. URL https://arxiv.org/abs/2508.05469.

Kayla Schroeder and Zach Wood-Doughty. Can you trust llm judgments? reliability of llm-as-a-judge, 2024. URL https://arxiv.org/abs/2412.12509.

Weixi Tong and Tianyi Zhang. Codejudge: Evaluating code generation with large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 20032–20051. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.emnlp-main.1118. URL https://arxiv.org/abs/2410.02184.

Haiming Wang, Mert Unsal, Xiaohan Lin, Mantas Baksys, Junqi Liu, Marco Dos Santos, Flood Sung, Marina Vinyes, Zhenzhe Ying, Zekai Zhu, Jianqiao Lu, Hugues de Saxcé, Bolton Bailey, Chendong Song, Chenjun Xiao, Dehao Zhang, Ebony Zhang, Frederick Pu, Han Zhu, Jiawei Liu,

Jonas Bayer, Julien Michel, Longhui Yu, Léo Dreyfus-Schmidt, Lewis Tunstall, Luigi Pagani, Moreira Machado, Pauline Bourigault, Ran Wang, Stanislas Polu, Thibaut Barroyer, Wen-Ding Li, Yazhe Niu, Yann Fleureau, Yangyang Hu, Zhouliang Yu, Zihan Wang, Zhilin Yang, Zhengying Liu, and Jia Li. Kimina-prover preview: Towards large formal reasoning models with reinforcement learning, 2025. URL https://arxiv.org/abs/2504.11354.

Kaiyu Yang, Gabriel Poesia, Jingxuan He, Wenda Li, Kristin Lauter, Swarat Chaudhuri, and Dawn Song. Formal mathematical reasoning: A new frontier in AI, 2024. URL https://arxiv.org/abs/2412.16075.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, pp. 46595–46623. Curran Associates, Inc., 2023. doi: 10.52202/075280-2020. URL https://arxiv.org/abs/2306.05685.

Terry Yue Zhuo. ICE-score: Instructing large language models to evaluate code. In Yvette Graham and Matthew Purver (eds.), *Findings of the Association for Computational Linguistics: EACL 2024*, pp. 2232–2242, St. Julian's, Malta, March 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-eacl.148. URL https://aclanthology.org/2024.findings-eacl.148/.

A JUDGE PROMPTS AND SCORING CONFIGURATION

The primary family model identifiers are `gpt-5.3-codex`, `claude-opus-4-7`, and `Qwen/Qwen3-Coder-30B-A3B-Instruct`. Each family panel contains the three semantic prompts below and `BAD_STRINGY`, plus `gpt-4o-mini` with the checklist and strict prompts. The other two configurations are the deterministic edit-distance control `editdist_e1` and seeded control `random_r1`. All model prompts request scores in $[0, 1]$, not integers on a 0–10 scale. The human ratings use 0–5; correlations compare their ordering with judge scores without equating the scales.

The reference field retains its historical label "REFERENCE IMPLEMENTATION," but the code fills it with the gold's final theorem block. The candidate field contains the full candidate file for human-agreement scoring; property probes operate on the extracted theorem text. If no theorem block is found, the extractor uses the full file. The Codex panel aggregates three recorded calls for model configurations and nine for controls by the median. The Opus and Qwen panels aggregate three calls for every configuration by the median. These counts come from the saved repeat arrays; a summary-level repeat setting alone does not describe reused cached rows. The random control is deterministic for its input and seed, so repeated identical scores do not imply a reliable semantic judge. Prompt labels P1, P4, and P8 are historical identifiers and do not denote the property numbers P1–P4.

P1_BASELINE

```
You are an expert in Lean 4 formal verification.
Score how well the GENERATED THEOREM captures the correctness properties
of the REFERENCE IMPLEMENTATION. Score from 0.0 to 1.0.

- 1.0: Semantically equivalent to what the reference guarantees
- 0.7-0.9: Captures most properties, misses minor ones
- 0.4-0.6: Captures some properties, has meaningful gaps
- 0.1-0.3: Technically valid Lean but misses core properties
- 0.0: Trivially true (e.g., `True`) or incorrect

REFERENCE IMPLEMENTATION:
{reference_impl}

GENERATED THEOREM:
{generated_theorem}

Respond ONLY with valid JSON, no markdown:
{"score": <float 0.0-1.0>,
 "reasoning": "<one sentence max>"}
```

P4_CHECKLIST

```
Evaluate this in order:
1) What properties does REFERENCE IMPLEMENTATION guarantee?
2) Which are preserved by GENERATED THEOREM?
3) Are there missing core guarantees or trivialization?
4) Map to score in [0.0, 1.0]:
   1.0 equivalent; 0.7-0.9 minor gaps;
   0.4-0.6 meaningful gaps; 0.1-0.3 weak;
   0.0 trivial/incorrect.

REFERENCE IMPLEMENTATION:
{reference_impl}

GENERATED THEOREM:
{generated_theorem}

Return JSON only:
{"score": <float 0.0-1.0>,
 "reasoning": "<one sentence>"}
```

P8_STRICT_B

```
Evaluate conservatively. If uncertain, choose the lower score.
Use [0.0,1.0]:
1.0 equivalent semantics
0.7-0.9 mostly right, minor misses
0.4-0.6 substantial omissions
0.1-0.3 weak relation
0.0 trivial/incorrect
```

```
REFERENCE IMPLEMENTATION:
{reference_impl}

GENERATED THEOREM:
{generated_theorem}

Return valid JSON only:
{"score": <float>,
 "reasoning": "<one sentence>"}
```

BAD_STRINGY negative control

```
You are a text similarity grader.
Compare generated theorem text to the reference implementation text.
Do NOT evaluate semantic correctness or theorem meaning.
Score ONLY based on superficial lexical/format overlap:
- high score if tokens/phrases look similar
- low score if wording/structure differs
Reference implementation:
{reference_impl}

Generated theorem:
{generated_theorem}

Return ONLY a JSON object:
{"score": <float 0.0 to 1.0>,
 "reasoning": "<short reason>"}
```

B Index Definitions and Implementation

Fixed equal-weight index. For each file f and bug level ℓ, let $s_{f\ell}$ be the median recorded repeat score and $v_{f\ell}$ its population variance. Let ρ_f be Spearman's correlation between level and $s_{f\ell}$, and set $c_f = \mathrm{clip}((1-\rho_f)/2, 0, 1)$. Writing $\mathcal{F}$ for the included ladders, the archived implementation uses

$$C_f = \frac{1}{|\mathcal{F}|}\sum_{f\in\mathcal{F}} c_f, \qquad V_c = \mathrm{Var}_{f\in\mathcal{F}}(c_f), \qquad V_s = \frac{\sum_{f\in\mathcal{F}}\sum_{\ell} v_{f\ell}}{\sum_{f\in\mathcal{F}} L_f},$$

where L_f is the number of levels in ladder f. It skips ladders with fewer than three levels or an undefined Spearman correlation, including constant-score ladders. All primary committed vectors report 24 included ladders per configuration. If none remain, the code substitutes $C_f = 0$, $V_c = 1$, and $V_s = 1$. It min–max normalizes each variance across the current family panel; a constant variance column maps to zero. Equation 1 then applies weights $1/3$ to C_f, $1-\widetilde{V}_c$, and $1-\widetilde{V}_s$, after clipping each to $[10^{-6}, 1]$. Thus an extreme variance does not produce an exact zero index, and adding a judge can change other judges' normalized scores. The analysis preserves these choices rather than changing the frozen index after observing results.

Arithmetic comparisons. The recorded $v1$ and $v2$ are

$$v1 = 0.30\,\mathrm{ident} + 0.30\,\mathrm{mono} + 0.25\,\mathrm{weak} + 0.15\,\mathrm{stab},$$
$$v2 = 0.20\,\mathrm{ident} + 0.20\,\mathrm{mono} + 0.15\,\mathrm{weak} + 0.10\,\mathrm{stab} + 0.20\,\mathrm{anti} + 0.15\,\mathrm{subtle}.$$

Here ident is mean score over identity-rewrite probes, $\mathrm{mono} = (1-\rho_{\mathrm{bug}})/2$ uses the pooled bug-level Spearman correlation, and weak is the weak-monotonicity fraction on bug ladders. The latter is the historical field `P3_weak_mono`; it contains no missing-specification measurement. stab is one minus the fraction of scored pairs whose repeated scores cross the artifact-score threshold. anti is the complement of the absolute score correlation with normalized edit distance, clipped to $[0,1]$, and subtle is the clipped mean first-level score drop divided by 0.25. The implementation clips these components to $[0,1]$. These arithmetic indices combine different summaries from $\mathrm{TI}_{\mathrm{var}}$ and do not isolate the effect of changing only the aggregation function.

Geometric comparisons. The two-component comparison in Table 2 is $\mathrm{GM}_{.4,.6} = \mathrm{mono}^{0.4}\mathrm{stab}^{0.6}$, with the same 10^{-6} component floor. Its weights originate in the pass1 search over six components; it is not a fixed equal-weight baseline. The separate cross-pass analysis searches nonnegative weights summing to one over ident, mono, weak, stab, anti, and subtle. Table 5 identifies which pass supplies those fitted weights. Neither procedure fits the weights of $\mathrm{TI}_{\mathrm{var}}$.

C Additional Validation-Set Results

C.1 A separately weight-tuned geometric variant

The cross-pass analysis fits a geometric combination of six available components on one label pass and evaluates it on the other. The selected pass1 weights are 0.4 for bug monotonicity and 0.6 for decision stability. Both passes concern the same candidate–gold pairs from one annotator, so the second pass is not a test set. This analysis uses its own frozen label-alignment output; it is not a re-estimation of the fixed equal-weight index.

Pass supplying weights	Association on that pass	Association on other pass
pass1	0.881	0.810
pass2	0.857	0.786

Table 5: The tuned geometric variant remains positively associated across two rating passes of the same 72 pairs. Entries are judge-level Spearman correlations; the fitted weights are separate from $\mathrm{TI}_{\mathrm{var}}$.

C.2 Artifact-score threshold

The historical threshold sweep uses the strict Codex prompt on 176 files with three repeated scores, aggregated by the mean rather than the headline panels' median. It thresholds artifact scores, not the index used to compare judges. The raw bug-level ranking is independent of this threshold and remains $\rho_{\mathrm{bug}} = -0.790$ throughout the sweep. The intervals in Table 6 describe the recorded sweep; they do not establish accepted-artifact correctness.

Threshold	Flip rate (95% interval)	Pass rate (95% interval)
0.60	0.058 $[0.048, 0.069]$	0.583 $[0.559, 0.608]$
0.70	0.049 $[0.040, 0.059]$	0.559 $[0.537, 0.581]$
0.80	0.051 $[0.042, 0.061]$	0.544 $[0.523, 0.566]$
0.85	0.046 $[0.037, 0.057]$	0.536 $[0.515, 0.557]$

Table 6: A stricter artifact-score threshold slightly reduces recorded decision flips and the fraction accepted. The table shows selected points from the historical sweep; 0.85 has its lowest observed flip rate.

C.3 Local weight sensitivity

We recompute the index from committed component vectors on a simplex grid with each weight in $\{0.2, 0.3, 0.4, 0.5\}$ and weights summing to one. This yields 12 triples around the equal-weight choice. The original family-specific normalization and 10^{-6} floor remain fixed. No judge is rescored and no replacement weights are selected from this grid. The script and its complete output are named in the Reproducibility Statement.

Family	pass1 range	pass2 range	avg range
Codex	$[0.810, 0.905]$	$[0.738, 0.810]$	$[0.738, 0.857]$
Claude Opus	$[0.786, 0.810]$	$[0.571, 0.690]$	$[0.643, 0.714]$
Qwen3-Coder	$[0.357, 0.762]$	$[0.738, 0.952]$	$[0.881, 0.976]$

Table 7: The association remains positive on this local weight grid, with the widest pass1 variation for Qwen. Ranges cover judge-level Spearman correlations across the 12 weight triples; they are sensitivity ranges, not confidence intervals.

C.4 JUDGE-LEVEL DIAGNOSTICS FROM THE COMMITTED INDEX VECTORS

Table 8 recomputes three descriptive diagnostics from the committed per-configuration vectors of each family panel, with no new judge calls. The exact permutation probability enumerates all $8!$ orderings of the human-agreement values and counts those whose absolute Spearman correlation with the index reaches the observed one; ties receive average ranks. The exact Codex values, 0.015, 0.005, and 0.046, agree with the archived resampled values in Table 2 to within 0.002. The component-only column correlates the bug-monotonicity term C_f with human agreement and matches the archived fields to three decimals. The subset columns drop the edit-distance and random controls, then also the two GPT-4o-mini prompts; they are descriptive, keep the original full-panel normalization, and carry no probabilities.

Family	Labels	ρ ($n = 8$)	exact p	$\rho(C_f)$ ($n = 8$)	ρ without controls ($n = 6$)	ρ same-model prompts ($n = 4$)
Codex	avg	0.833	0.015	0.643	0.771	0.400
Codex	pass1	0.905	0.005	0.690	0.886	0.800
Codex	pass2	0.738	0.046	0.667	0.657	0.000
Claude Opus	avg	0.643	0.096	0.467	0.486	0.000
Claude Opus	pass1	0.810	0.022	0.527	0.829	0.600
Claude Opus	pass2	0.595	0.132	0.575	0.486	0.000
Qwen3-Coder	avg	0.905	0.005	0.690	0.943	0.800
Qwen3-Coder	pass1	0.476	0.243	0.048	0.600	0.600
Qwen3-Coder	pass2	0.857	0.011	0.905	0.943	0.800

Table 8: The association stays positive after removing the non-LLM controls but is coarse among same-model prompts. Entries are Spearman correlations between the fixed index (or its bug component) and judge-level human agreement, recomputed from committed vectors; the nine exact probabilities are unadjusted.

D HUMAN AGREEMENT AND ADDITIONAL JUDGE PANELS

D.1 AGREEMENT BETWEEN ANNOTATION PASSES

Table 9 reports the frozen agreement statistics on 72 shared analyzed pairs. The passes reflect different rubric strictness levels from the same annotator, rather than independent draws from a rater population. Their disagreement cannot identify a numerical ceiling on model agreement. The original rubric text remains unavailable.

Statistic	Estimate	95% bootstrap interval
Spearman correlation	0.249	$[-0.015, 0.489]$
Pearson correlation	0.375	$[0.076, 0.604]$
Kendall correlation	0.221	$[-0.016, 0.442]$
Quadratic-weighted κ	0.372	$[0.073, 0.581]$
Mean absolute error	0.771	Not reported

Table 9: The two human passes disagree on the same artifacts, making the choice of label view consequential. Mean absolute error uses the original 0–5 rating scale; all estimates use 72 overlapping pairs.

D.2 EXPANDED CONFIGURATION PANEL

The expanded panel compares the arithmetic $v2$ with the pass1-selected geometric form $\mathrm{mono}^{0.4}\mathrm{stab}^{0.6}$. Its averaged-label association is 0.769, with bootstrap interval $[0.34, 0.95]$ (Figure 4). The result concerns 13 configurations on reused validation labels, not 13 independent model families or unseen data. The plotted geometric index is not $\mathrm{TI}_{\mathrm{var}}$.

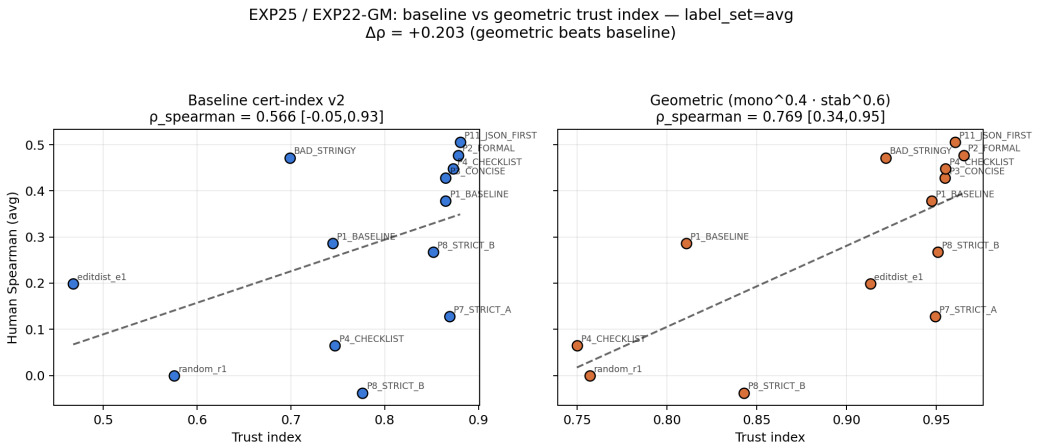


Figure 4: The tuned geometric comparison has a positive association on the expanded validation panel. The two panels compare arithmetic $v2$ with $\mathrm{mono}^{0.4}\mathrm{stab}^{0.6}$ across 13 configurations; the figure retains the archived estimates and intervals.

D.3 Negative results

A separate Leanstral panel yields $\mathrm{TI}_{\mathrm{var}}$ associations of -0.1190, -0.0476, and -0.0952 on pass1, pass2, and averaged labels. These recorded results contradict a claim that the fixed index works for every family. They do not establish why the association fails. The different-rubric human pool described in Section 6 also produced no qualifying index in either registered screen. The 32-configuration screen's largest absolute association is negative, so its sign does not support the proposed direction of the screen even before accounting for multiple comparisons.

E Property Diagnostics and Probe Validity

Historical generator limitations. The recorded experiments use string-edit generators without Lean elaboration checks. The alpha routine rewrites names in the theorem signature, leaves proof-body references unchanged, and can duplicate the `:=` marker. Consequently, its low scores cannot establish failure of invariance under valid alpha-renaming. The renaming and reordering routines also lack checks against variable capture and dependent-binder errors. The bug ladder changes relations and connectives, removes a theorem block, and introduces trivial statements; unavailable edits fall back to comments. Thus cumulative edit level is an intended damage order, not a formally verified ordering of semantic quality. These limitations apply to the archived property statistics throughout the paper.

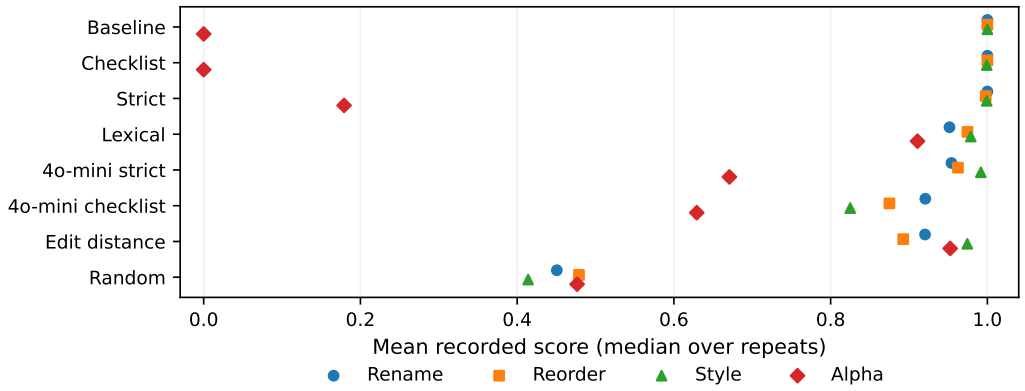


Figure 5: Recorded alpha scores differ sharply from the other rewrite scores for the semantic Codex prompts. Each marker averages repeat-median scores over 24 problems; these unvalidated transformations do not establish semantic sensitivity.

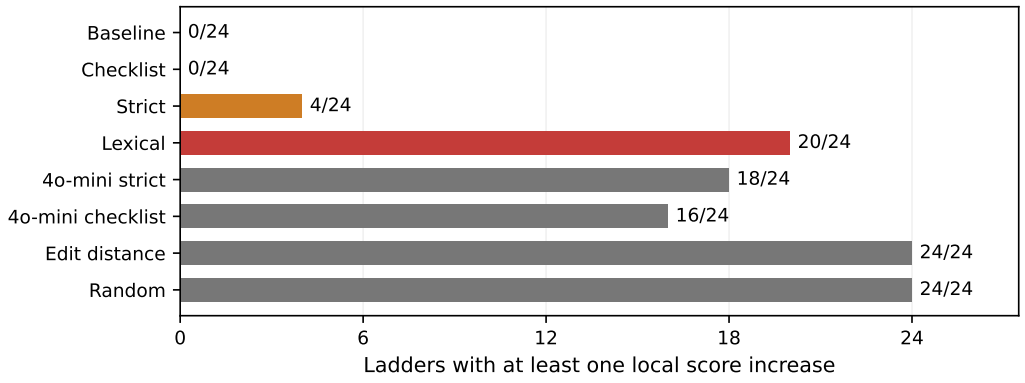


Figure 6: Local increases reveal behavior hidden by the aggregate bug trend. Bars count ladders with at least one increase in the recorded repeat-aggregated score; all configurations have 24 ladders, matching Table 3.

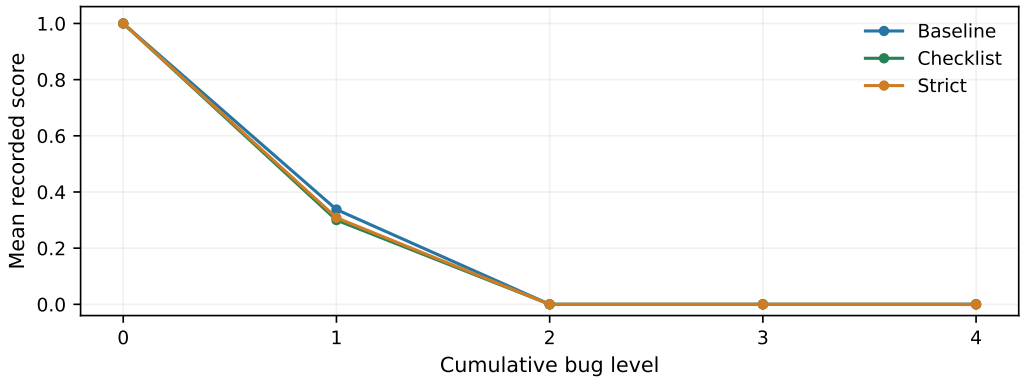


Figure 7: Claude Opus also assigns lower mean scores as historical bug edits accumulate. Each point averages repeat-median scores over 24 problems under the same unvalidated string-edit construction; its trend does not establish semantic validity of every ladder.

F Unit Tests as Ground Theorems: A Definition We Do Not Adopt

Section 3 follows common formal-methods usage: the formal specification consists of theorem statements about the implementation, and unit tests are not part of it. We record a competing view that we do not adopt. In Lean 4, a unit test is often itself a closed theorem. The gold `myAdd` artifact states its tests as, e.g., `example : myAdd 1 2 = 3 := by native_decide`, an anonymous theorem with no free variables. On this view, a unit test specifies the implementation at one concrete input, and specifications range from such ground instances to universally quantified properties such as $\forall n,\ \texttt{myAdd}\ n\ 0 = n$. The choice changes terminology, not our measurements. The P3 ladder removes theorem statements and unit tests alike; under the adopted definition, only the removed theorems count as missing specification.